

Mizan.ai — Modal GPU Cold-Start Investigation

QwenBrain (A10) & QwenLite (T4) serving classes · modal-serving/modal_app.py · 2026-07-25

1. Background & Goal

Earlier RAG/chat testing surfaced intermittent **RemoteDisconnected** errors traced back to GPU cold-start latency on Modal's serverless containers. Two vLLM-serving classes were in scope: **QwenBrain** (Qwen2.5-14B-Instruct GPTQ-Int4, A10 GPU, agent brain) and **QwenLite** (Qwen2.5-7B-Instruct GPTQ-Int4, T4 GPU, free-tier chat/RAG). Translator (MADLAD-400) and Embedder (BGE-M3) were explicitly out of scope — smaller, less cold-start-sensitive models.

Goal: reduce the wall-clock time a user waits when a request lands on a scaled-to-zero container, without paying for GPU capacity 24/7 (an always-warm `min_containers=1` approach was explicitly rejected on cost grounds early in this investigation).

1.1 Baseline (no changes)

Model	GPU	Cold start (end-to-end)
QwenLite	T4	141.2 s
QwenBrain	A10	132.9 s

2. Approach 1 — Modal GPU Memory Snapshot (attempted, reverted)

A handoff document ([docs/mizan_gpu_snapshot_handoff.pdf](#)) proposed Modal's experimental GPU memory snapshot feature: `split @modal.enter(snap=True) / @modal.enter(snap=False)` lifecycle hooks, `enable_memory_snapshot=True`, `experimental_options={'enable_gpu_snapshot': True}`, combined with vLLM's native sleep mode (`enable_sleep_mode=True`, `llm.sleep(level=1)` / `llm.wake_up()`).

2.1 Compatibility verification (done before writing any code)

- Confirmed via `inspect.signature()` against the installed Modal SDK (v1.5.2) that both `enable_memory_snapshot/experimental_options` and the `snap` parameter are genuinely supported.
- Modal's own reference examples (`llm_snapshot.py`, `ministral3_inference`) use a subprocess `vllm serve + HTTP /sleep//wake_up` pattern — a real architecture mismatch against this repo's in-process LLM class usage.
- Independently verified via vLLM's own sleep-mode docs that the offline LLM class supports native `.sleep()/wake_up()` Python methods directly — no subprocess rewrite was actually needed.

2.2 What happened

Implemented and deployed for both classes. Logs confirmed our code executed correctly end-to-end: engine load, `torch.compile`, CUDA graph capture, two warmup requests, and `sleep(level=1)` all succeeded. **Modal's own infrastructure-level snapshot-creation step then failed** ("Failed creating Function memory snapshot"), and containers began repeatedly failing to start. Modal sent an automated alert email confirming the app was failing to start containers.

Root cause: an experimental Modal platform feature failing at the infrastructure level, after application code had already done everything correctly. Not fixable from application code. Per the handoff doc's own guidance,

correctness must not depend on this feature succeeding.

2.3 Resolution

User instruction: revert everything and restore stable service; explicitly do not pursue always-warm (min_containers=1) as a workaround, on cost grounds. Both classes were reverted to their original single-@modal.enter() structure. Verified via a live test that cold-start behavior matched the pre-experiment baseline almost exactly (140.7 s vs. 141.2 s baseline). The vllm==0.22.1 version pin added during this work was kept — independently useful, since sleep-mode-adjacent APIs have moved across vLLM versions before.

3. Approach 2 — Two Non-Experimental Techniques

Two well-established, non-experimental vLLM-level techniques were tried instead, each in isolation, starting with QwenLite before extending to QwenBrain:

- **Experiment A — enforce_eager=True:** disables both torch.compile and CUDA graph capture (equivalent to CompilationConfig(mode=NONE, cudagraph_mode=NONE)). Trades ~10-15% steady-state throughput for a much faster boot.
- **Experiment B — VLLM_CACHE_ROOT on the Modal Volume:** points vLLM's torch.compile cache directory at a path on the same persistent Volume the model weights already live on (default is ephemeral container-local disk, wiped every cold start).

Methodology note: an early reading of Experiment A on QwenLite showed ~6.8 s end-to-end. That number did not survive later scrutiny (Section 5) and is superseded by the re-measured figures below, which separate Modal's own container-provisioning overhead from vLLM's actual engine-boot time.

3.1 Experiment A results — enforce_eager=True

Model	Metric	Baseline	enforce_eager=True	Change
QwenLite	vLLM engine init	80.48 s (compile incl.)	17.2 – 19.1 s (no compile)	~4-5x
QwenLite	End-to-end cold start	141.2 s	~68 – 73 s	~2x
QwenBrain	vLLM engine init	72.53 s (compile: 46.85 s)	14.82 s (no compile)	~5x
QwenBrain	End-to-end cold start	115.79 s*	64.47 s	~1.8x

* This particular baseline run (115.79 s) is slightly faster than the original 132.9 s baseline capture — normal run-to-run variance in Modal's own provisioning latency, not a code difference.

3.2 Experiment B results — VLLM_CACHE_ROOT (QwenLite only)

Tested across three separate cold starts as the persisted cache matured:

Run	Compile time	Engine init total	End-to-end
1st (cache-creating)	46.59 s	94.09 s	—
2nd (genuine scale-to-zero, cache reused)	30.26 s	71.91 s	94.6 s
3rd (re-deploy, cache still present)	15.05 s	51.67 s	90.55 s

The cache genuinely gets reused and gets more effective with repeated hits — but **CUDA graph capture (~16-23 s) still runs every cold start**, since only the compile step is cached. Best observed end-to-end: ~90.55 s vs. 141.2 s baseline (~1.6x).

The two don't combine. enforce_eager=True skips torch.compile entirely, so there is no compile artifact for VLLM_CACHE_ROOT to cache — setting both together is a silent no-op on the caching half.

4. Final Decision

enforce_eager=True, applied to both QwenLite and QwenBrain. Reasoning: faster end-to-end than the cache-persistence approach in every measurement, and structurally simpler — no Volume cache-management, no cache-invalidation concerns across vLLM/model updates.

Model	Baseline	Final (enforce_eager=True)	Improvement
QwenLite	141.2 s	~68 – 73 s	~2x
QwenBrain	115.79 – 132.9 s	64.47 s	~1.8 – 2x

Accepted trade-off: ~10-15% lower steady-state decode throughput per vLLM's own documentation, in exchange for eliminating both the torch.compile and CUDA graph capture cost on every cold start.

5. Methodology Correction: What "Clean Cold Start" Actually Means

Several early readings across this investigation turned out to be misleading because they hit a still-warm container rather than a genuine cold start (observed false-fast readings: 4.1 s, 3.75 s, and the original ~6.8 s Experiment A figure). The fix adopted partway through: always confirm modal container list --json returns an empty list immediately before firing a timed test request, and cross-check the response against a genuinely new EngineCore pid= log line in the same timeframe.

A second, larger correction came later: **total end-to-end latency has two distinct layers**, and only one of them is something these experiments can affect.

Layer	What it covers	Affected by enforce_eager?
Modal container provisioning	GPU allocation, image pull/mount, process start, before our code runs at all	No — consistently ~35-50 s regardless of config
vLLM engine boot	Weight load, (optionally) torch.compile + CUDA graph capture, KV cache init	Yes — this is what the experiments target

Modal reports this split directly in its own logs as duration (total) vs. execution (just the request-handler's own time) — the gap between them is provisioning + engine-boot combined. Cross-checking the original "~6.8 s" Experiment A reading against this: vLLM's own init engine line consistently needs ≥ 17 s on its own even under eager mode, which is already larger than the entire original reading — proof that reading was not a genuinely clean cold start, despite the verification attempted at the time. The re-measured ~68-73 s / ~64.47 s figures in Section 3 are the ones to trust; the provisioning-layer overhead is currently outside what any vLLM-level change can reduce.

6. Side Investigation: Recurring Container Restarts

During testing, containers were repeatedly observed restarting shortly after being manually killed (dashboard/CLI), including one reboot only **27 seconds** after the prior container's shutdown, with no request sent by either the user or this session in that window.

6.1 Ruled out

- Application code: full review of modal_app.py and common/model_loader.py — no min_containers, no keep_warm, no background threads, loops, or schedulers anywhere.
- Repo-wide search for cron/scheduler/keep-alive patterns across the whole codebase — none found.
- A duplicate app = modal.App("mizan-models") definition was found at app/modal_app.py (stale, last edited Jul 14) alongside the actively-maintained modal-serving/modal_app.py. Deploy history timing suggested the stale file may have been deployed once, days earlier — but four subsequent same-day deploys from the correct file fully superseded it, ruling it out as the cause of same-day restarts.
- User-side monitoring, uptime checkers, or an open dashboard tab — user confirmed none.

6.2 What actually stopped it

Killing individual containers (dashboard or modal container stop) did not stop the pattern — the app remained deployed and reachable, so something could always bring a new container up. **Stopping the entire app** from the dashboard ("Stop App", distinct from killing a single container) did work — confirmed via an extended, genuinely quiet period with zero containers and no reboot afterward.

Open question, not fully resolved: the exact source of the periodic reboot behavior while the app is deployed was not conclusively identified. Most likely explanation: Modal's own platform-side behavior (post-deploy validation warm-up and/or periodic health-check traffic against web endpoints), invisible to application code or

account-level settings. If this recurs in a way that matters for cost, it is worth raising directly with Modal support, with the timestamped log evidence gathered here.

7. CUDA & Triton Concepts Encountered

Captured for reference, as background for further CUDA/Triton learning.

7.1 torch.compile pipeline

- **Dynamo:** traces Python bytecode into an FX graph representation of the model's forward pass.
- **Inductor:** compiles that graph into optimized kernels (fusing operations, generating Triton kernels where applicable). This is the expensive step — the bulk of the 15-47 s "compilation" time measured throughout this investigation.
- `enforce_eager=True` is equivalent to `CompilationConfig(mode=CompilationMode.NONE, cudagraph_mode=CUDAGraphMode.NONE)` — it skips both Dynamo tracing and Inductor compilation entirely, running standard eager-mode PyTorch ops instead.

7.2 CUDA graph capture

A separate step from `torch.compile`: records a fixed sequence of GPU kernel launches once, then replays the whole sequence with a single CPU dispatch on later calls, avoiding per-kernel Python/CPU launch overhead. Observed taking ~16-23 s across the compiled (non-eager) runs in this investigation (logged as "Capturing CUDA graphs (mixed prefill-decode, PIECEWISE)" and "(decode, FULL)" phases). Also skipped by `enforce_eager=True`.

7.3 Triton

- A Python-like GPU kernel-authoring language that JIT-compiles down to PTX/CUDA — used here as vLLM's `TRITON_ATTN` attention backend on QwenLite's T4 GPU, specifically to avoid FlashInfer's default sampler, which JIT-compiles via `nvcc` (not present in this repo's slim container image, causing a hard "Could not find nvcc" crash on first sampled request).
- Distinct, smaller JIT cost observed at first-request time for shapes not covered by warmup: `_compute_slot_mapping_kernel`, `kernel_unified_attention`, `reduce_segments` — each logged as its own latency-spike warning, separate from the larger `torch.compile/CUDA-graph` cost.

7.4 GPTQ-Int4 quantization + Marlin kernel

Both models run GPTQ-Int4 checkpoints. vLLM selects `MarlinLinearKernel` (`AutoGPTQLinearMethod`) — a specialized fused INT4 GEMM kernel — for the quantized linear layers, visible in the logs as `Using MarlinLinearKernel for AutoGPTQLinearMethod`.

7.5 vLLM V1 engine architecture

- Model serving runs in a separate `EngineCore` subprocess from the main container process.
- KV cache size is computed dynamically from available GPU memory at startup (e.g. "GPU KV cache size: 125,520 tokens" on QwenLite's T4).
- Chunked prefill scheduling batches large prompts into smaller chunks alongside decode steps ("Chunked prefill is enabled with `max_num_batched_tokens=8192`").

7.6 GPU memory snapshot vs. vLLM sleep mode

Modal's experimental GPU snapshot feature is conceptually closer to a CRIU-style full process/GPU-state snapshot-and-restore. vLLM's own native sleep mode (`llm.sleep(level=1)`) is lighter-weight by design: it

discards the KV cache (cheaper to rebuild than to snapshot, per vLLM's own guidance) rather than freezing the full process state. The snapshot approach in Section 2 combined both — sleep to shrink GPU memory footprint, then Modal's own infrastructure to snapshot what remained — which is where the infra-level failure occurred.

8. Files Changed

- **modal-serving/modal_app.py** — vllm==0.22.1 pinned; both QwenBrain and QwenLite now call `load_vllm_engine(..., enforce_eager=True)`; QwenLite additionally forces `AttentionConfig(backend="TRITON_ATTN")` (pre-existing, unrelated to this investigation). No GPU-snapshot or cache-persistence code remains.
- **modal-serving/common/model_loader.py** — unchanged from its original signature; briefly gained an `enable_sleep_mode` parameter during the snapshot attempt, fully reverted.